

ValeCare Frontend Features Specification

React and Vite Implementation

Prepared for the SRHR Digital Solutions Project

Overview

ValeCare gives adolescent girls and young women a private way to reach sexual and reproductive health information and services from a phone. It is built in React and Vite, and it leans on optional accounts, offline support, and a light, low bandwidth experience so it holds up on slow connections and shared devices.

Every feature is fully usable without creating an account. The only thing an account adds is a saved history, chat sessions, bookmarks, and cycle or pregnancy logs, that follows a user from one visit to the next.

Technology Stack

- React 19 with Vite, React Router DOM for routing, and the React Context API for state
- Axios for API calls, React Hook Form and Yup for forms and validation
- React i18next for translation across English, French, and Portuguese, alongside a broad set of languages native to Africa: Swahili, Igbo, Yoruba, Hausa, Amharic, Zulu, Xhosa, Shona, Somali, Twi, Wolof, Lingala, Kinyarwanda, and Afrikaans
- Framer Motion and Lottie React for animation, React Icons and Recharts for icons and charts
- IndexedDB and localStorage for offline and cached data, Leaflet for maps
- vite-plugin-pwa so the app installs and keeps working offline as a Progressive Web App

Design Principles

- Privacy by design: nothing SRHR specific appears in notifications, browser titles, or lock screen previews.
- Guest by default, accounts by choice: every feature works fully without an account. Signing up only adds a saved history.
- Open access: all verified content is available to every user. There is no age gate or age based filtering anywhere in the app.
- Mobile first, with desktop and tablet layouts added after.
- Low bandwidth: lazy loading, compressed images, and local caching throughout.
- Accessible to WCAG 2.1 AA.
- Plain language a teenager can follow, not clinical terminology.

Global Layout

Every page shares a small set of persistent pieces: an App Header with logo and language selector, a Main Content Area, a Floating Emergency Action Button that stays one tap away from anywhere in the app, a Bottom Navigation Bar on mobile, and a Side Navigation on tablet and desktop.

Feature 1: Welcome and Onboarding

Serves REQ-1.1, REQ-1.2, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: accounts app.

This is the first screen a user sees. It introduces the app, reassures people about privacy, lets them pick a language, and offers three ways forward: continue as a guest, sign in to an existing account, or create an account.

Key screens and components

- Welcome page: logo, tagline, short about card, language selector, privacy card, and the three entry buttons.

- Privacy card explaining encryption, anonymous browsing, and user control in plain terms, with a link to the full privacy policy.

Backend dependency

The language list and privacy policy version load from the API on first launch. Every one of the three entry buttons calls the accounts app, either to start a guest session or to move into sign up or sign in.

Feature 2: Account and Guest Access

Serves REQ-1.2, REQ-2.1, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: accounts app.

A user can use every part of the app without creating an account. Choosing Continue as Guest generates a device token and nothing more. She gets full access to information, the AI assistant, provider search, and every other feature, and once she leaves the app that session is gone.

Choosing Create Account starts a short sign up: a username and a passphrase, nothing else. No name, phone number, or email is collected. At the end of sign up, the app shows a one time recovery code and asks her to save it somewhere safe, since it is the only way to reset a forgotten passphrase later. The code is shown once and never shown again after that screen.

Choosing Sign In asks for the username and passphrase and links the current device to that account. If she forgets her passphrase, a Recover Access screen asks for her username and her recovery code, lets her set a new passphrase, and issues a new recovery code to save.

Key screens

- Continue as Guest: automatic, no form.
- Create Account: username, passphrase, passphrase confirmation, then a one time recovery code screen with a clear prompt to save it.
- Sign In: username and passphrase.
- Recover Access: username, recovery code, new passphrase.

Local session storage

- device_token, account_type, preferred_language, theme, last_active

API

- POST /api/accounts/session/, starts or restores a guest session
- POST /api/accounts/signup/, creates a registered account and returns the one time recovery code
- POST /api/accounts/login/, signs in and links the device to the account
- POST /api/accounts/recover/, resets the passphrase using the recovery code and issues a new one
- PATCH /api/accounts/profile/, updates language, username, or passphrase

Feature 3: Home Dashboard

Serves REQ-1.1, REQ-1.2, REQ-NF-2. Backend: draws from several apps.

The landing page after entry. It surfaces recommended learning content, nearby providers, open follow up reminders, and a daily tip. For a registered user it also links back into her saved history; for a guest it shows the same layout without a history section.

API

- GET /api/education/content/?recommended=true
- GET /api/providers/?lat=&lng=&max_distance_km=
- GET /api/followup/tasks/
- GET /api/emergency/guides/sync/?since_version=

Feature 4: Education and Learning Module

Serves REQ-1.1, REQ-1.2, REQ-2.1, REQ-NF-2, REQ-NF-3. Backend: education app.

A learning hub for browsing, searching, and reading SRHR content: articles, short quizzes, infographics, and frequently asked questions, filtered by language and category, with every topic open to every user.

API

- GET /api/education/content/?category=&language=
- GET /api/education/content/{id}/
- POST /api/education/bookmarks/, GET /api/education/bookmarks/, DELETE /api/education/bookmarks/{id}/
- POST /api/education/progress/, GET /api/education/progress/

Bookmarks and reading progress are saved for registered users and follow them to any device they sign in on. A guest can read and use everything in this section, the bookmark and progress icons are simply not shown without an account, since there is nowhere to save them once the session ends.

Feature 5: AI Health Assistant and Symptom Checker

Serves REQ-1.3, REQ-2.1, REQ-2.2, REQ-2.3, REQ-3.5, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: triage app.

A conversational assistant for SRHR questions, paired with a symptom checker that walks a user through follow up questions and points them toward the right next step, whether that is self care information or an urgent referral.

API

- POST /api/triage/sessions/
- POST /api/triage/sessions/{id}/messages/
- GET /api/triage/sessions/{id}/messages/
- DELETE /api/triage/sessions/{id}/
- POST /api/triage/symptoms/

Every assistant message from the backend carries an `is_verified_source` flag on the `TriageMessage` record. The trust badge shown next to each reply reads directly from that flag, which is what answers the distrust adolescents tend to have toward unverified AI answers.

For a registered user, past sessions stay in her history and are reachable from the dashboard. For a guest, the conversation is available for the length of the visit and is not saved anywhere once she leaves.

Feature 6: Provider Search and Appointment Booking

Serves REQ-3.1, REQ-3.2, REQ-3.3, REQ-NF-2, REQ-NF-3. Backend: providers and appointments apps.

A directory of verified, youth friendly providers with filters for gender preference, distance, and faith sensitivity, plus a simple booking flow that shows a reference code instead of the user's name.

API

- GET /api/providers/?gender=&max_distance_km=&faith_sensitive=&lat=&lng=
- POST /api/providers/{id}/reviews/
- POST /api/appointments/, GET /api/appointments/, PATCH /api/appointments/{id}/

Distance filtering on the backend is a simple haversine calculation against the `max_distance_km` parameter, not a geospatial database query. The map view on the frontend can still let a user drag a radius, it should just translate that into the same query parameter.

Feature 7: Counselor Support and Emergency Assistance

Serves REQ-4.1, REQ-4.2, REQ-NF-1, REQ-NF-2. Backend: support and emergency apps.

Each user can reach one trusted safe contact, such as a school counselor, through a single ongoing chat thread. The feature is built around one steady contact rather than a directory of counselors with specialties, ratings, and a booking calendar, so it stays about safety and continuity rather than shopping around.

For emergency guidance, the app fetches the full emergency guide set once and caches it locally, then refreshes only when the version number on the server changes. This is what keeps the guidance available even when a user's connection drops.

API

- GET /api/emergency/guides/sync/?since_version=

Feature 8: Menstrual Cycle Tracker

Serves REQ-5.1 through REQ-5.5, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: tracker app.

Lets a user log periods, flow, symptoms, and moods, and see predictions for her next cycle and fertile window.

API

- POST /api/tracker/cycles/, GET /api/tracker/cycles/, DELETE /api/tracker/cycles/{id}/
- GET /api/tracker/cycles/predictions/

Only a registered user's entries are saved to the server and carried between visits. A guest can use the tracker for the length of her session, those entries stay on the device and are cleared when she leaves.

Feature 9: Pregnancy, Fertility and Family Planning

Serves REQ-6.1 through REQ-6.4, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: education and tracker apps.

Combines educational content on pregnancy, fertility awareness, and contraceptive options with a personal pregnancy journal and week by week tracker.

API

- GET /api/education/content/?category=pregnancy
- GET /api/education/content/?category=family_planning
- POST /api/tracker/pregnancy/, GET /api/tracker/pregnancy/

The educational content is open to every user, guest or registered. The journal and week by week tracker follow the same rule as the cycle tracker: saved for registered users, session only for guests.

Feature 10: Notifications, Profile, Settings, and Account Management

Serves REQ-7.1 through REQ-7.5, REQ-NF-1, REQ-NF-2, REQ-NF-3. Backend: accounts app.

A guest sees a light version of this screen: change language, clear locally stored data, and a link to the privacy policy. A registered user sees all of that plus notification preferences, the option to change her username or passphrase, and the option to delete her account.

Deleting an account starts with a confirmation step. The app asks her to re-enter her passphrase, then shows a final warning that this removes her saved history, chat sessions, bookmarks, and cycle or pregnancy logs, and cannot be undone, before the deletion goes through.

API

- GET /api/accounts/profile/, PATCH /api/accounts/profile/
- POST /api/accounts/delete-request/, verifies the passphrase and returns a short lived confirmation token
- DELETE /api/accounts/profile/?confirmation_token=, finalizes the deletion

Cross-Cutting Frontend Standards

The points below apply to every feature above, so they are written once here instead of being repeated screen by screen.

Accessibility

- WCAG 2.1 AA color contrast, full keyboard navigation, and screen reader labels on every interactive element.
- Touch targets at least 48 by 48 pixels, and respect for the system's reduced motion setting.

Performance

- Route based code splitting, lazy loaded images, and compressed assets.
- Skeleton loading states instead of blank screens while data loads.

Privacy

- No SRHR specific text in push notifications, a neutral app icon, and sensitive screens kept out of the OS app switcher preview where the platform allows it.
- Recovery codes and passphrases are shown to the user once each, never emailed, texted, or logged anywhere the frontend can retrieve them later.

Offline behavior

- Emergency guides and previously viewed education content are cached through IndexedDB for offline use.
- Live features (chat, provider search, booking) need a connection and show a plain offline banner when unavailable, rather than failing silently.

Errors and empty states

- Every list screen needs a short, plain language empty state and a retry option on failure.
- No technical error codes shown to the user, keep the tone consistent with the rest of the app.